

ITA0606 - Machine Learning Short Answer Test

Q1. Perceptron Convergence Issues

A perceptron should converge for linearly separable data. Failure may occur due to an excessively high or low learning rate, incorrect weight initialization, missing or incorrect bias term, or an incorrect weight update rule. Debugging includes verifying the update equation, checking training data labels, monitoring weight changes, and plotting the decision boundary. Optimization techniques include proper feature normalization, random weight initialization, adaptive learning rates, and shuffling training samples to achieve faster and stable convergence.

Q2. Backpropagation Debugging

Slow convergence may result from vanishing gradients, sigmoid saturation, poor initialization, or an unsuitable learning rate. Debug by monitoring loss curves, checking gradients layer-wise, verifying forward/backward propagation, and testing on a small dataset. Improve performance using ReLU activation, Xavier/He initialization, batch normalization, Adam optimizer, and learning-rate scheduling.

Q5. Decision Tree Overfitting

High training accuracy with poor test accuracy indicates overfitting. Causes include deep trees, noisy data, and irrelevant features. Debug by comparing train/test accuracy, inspecting tree depth, and validating feature importance. Improve generalization through pre-pruning, post-pruning, cross-validation, feature selection, and removing noisy samples.

Q9. SVM on Nonlinear Data

Poor performance may be caused by selecting a linear kernel for nonlinear data, incorrect C or gamma values, or missing feature scaling. Debug by visualizing data, testing multiple kernels, and checking scaling. Optimize using the RBF kernel, Grid Search for hyperparameter tuning, normalization, and cross-validation to improve classification accuracy.

Q13. RNN Time-Series Prediction

Inconsistent predictions arise from vanishing gradients, short sequence lengths, or weak architecture design. Debug by monitoring training loss, gradient values, and hidden-state behavior. Replace standard RNNs with LSTM or GRU networks, apply gradient clipping, normalize sequences, and tune batch size and learning rate to improve long-term dependency learning.